UDF – CENTRO UNIVERSITÁRIO DO DISTRITO FEDERAL

Ciência da Computação

PROJETO CONECTAUDF: REDES SOCIAIS COM GRAFOS

Representação, métricas e recomendação de amizades em Java

Alunos: Arthur Santos Aguiar | Matheus Lopes Rodrigues | Rafael Ramos Magalhães
Curso: Ciência da Computação
Disciplina: Estruturas de Dados II
Professor(a): Kadidja Valéria
Data: 23/10/2026

1 INTRODUÇÃO

Este trabalho analisa a rede social fictícia ConectaUDF, formada por 6 usuários e 8 amizades. Como a amizade é recíproca, a rede foi modelada como um **grafo simples não direcionado**, em que cada vértice representa um usuário e cada aresta representa uma amizade.

O objetivo é desenhar o grafo, calcular o grau de cada usuário, o número de arestas e a densidade, encontrar caminhos mínimos e propor um critério de recomendação de amizades. O caderno do estudante sugeria Python; neste trabalho, todos os cálculos foram implementados em **Java**, e o código-fonte completo está no Apêndice A.

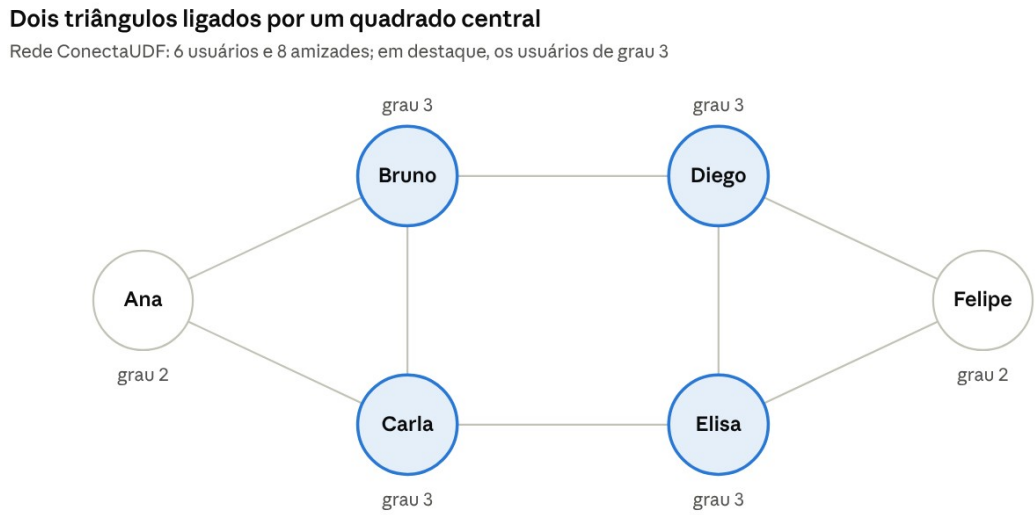
Em relação ao código-modelo do caderno, foram tomadas três decisões de implementação:

a) a lista de adjacência usa `Map<String, Set<String>>`, de modo que o `Set` impede amizades duplicadas;
b) o método `adicionarAmizade(a, b)` grava a aresta nos dois sentidos, evitando esquecer um dos lados ao digitar a rede;
c) foram implementados também a busca em largura (BFS), para encontrar todos os caminhos mínimos, e a recomendação de amizades por amigos em comum.

2 DESENHO DO GRAFO

O grafo possui 6 vértices e 8 arestas, uma para cada amizade da lista, sem nenhuma linha repetida. A Figura 1 apresenta o desenho da rede.

Figura 1 – Grafo da rede ConectaUDF



Fonte: elaborado pelo autor (2026).

Ao desenhar, percebe-se que a rede é simétrica: há dois triângulos nas extremidades (Ana, Bruno e Carla; Diego, Elisa e Felipe), ligados por um quadrado central formado por Bruno, Diego, Elisa e Carla. Os quatro usuários do quadrado são os de maior grau.

3 GRAU DOS USUÁRIOS

O grau de um vértice é a quantidade de arestas que incidem sobre ele, ou seja, o número de amigos diretos do usuário. A Tabela 1 apresenta os graus contados no desenho e confirmados pelo programa.

Tabela 1 – Graus dos usuários da rede original

Usuário	Amigos diretos	Grau
Ana	Bruno, Carla	2
Bruno	Ana, Carla, Diego	3
Carla	Ana, Bruno, Elisa	3
Diego	Bruno, Elisa, Felipe	3
Elisa	Carla, Diego, Felipe	3
Felipe	Diego, Elisa	2

Fonte: elaborado pelo autor (2026).

Pergunta 1: quem tem mais conexões diretas? Quatro usuários empatam com grau 3: Bruno, Carla, Diego e Elisa. Os quatro formam o quadrado central e fazem a ponte entre os dois triângulos. Ana e Felipe, nas extremidades, têm grau 2.

4 NÚMERO DE AMIZADES

Como cada aresta liga dois vértices, ela é contada uma vez em cada extremidade. Assim, a soma dos graus é sempre o dobro do número de arestas, resultado conhecido como lema do aperto de mãos. O número de arestas é dado pela Equação 1.

$$E = \Sigma \text{ grau}(v) / 2 = (2 + 3 + 3 + 3 + 3 + 2) / 2 = 16 / 2 = 8 \quad (1)$$

Portanto, a soma dos graus é 16 e, dividindo por 2, obtém-se **8 amizades**, exatamente as listadas no caderno.

5 DENSIDADE

A densidade compara as amizades existentes com o total de amizades possíveis. Para um grafo simples não direcionado, ela é calculada pela Equação 2.

$$D = 2E / [V \times (V - 1)] = (2 \times 8) / [6 \times (6 - 1)] = 16 / 30 \approx 0{,}5333 \quad (2)$$

O resultado decimal é 0,5333 e o resultado percentual é **53,33%**. Em um grafo com 6 vértices cabem no máximo C(6,2) = 15 amizades, e a rede possui 8, ou seja, pouco mais da metade.

Conclui-se que a rede é razoavelmente densa para o seu tamanho, mas está longe de um grafo completo (100%), em que todos seriam amigos de todos. Por exemplo, Ana e Felipe não são amigos e estão a três passos de distância.

6 CAMINHO DE ANA ATÉ FELIPE

O menor caminho de Ana até Felipe tem **3 arestas**, e existem dois caminhos com esse comprimento:

a) Ana → Bruno → Diego → Felipe;

b) Ana → Carla → Elisa → Felipe.

Para encontrar os caminhos foi utilizada a busca em largura (BFS). Em grafos sem peso, a BFS visita os vértices em ordem crescente de distância, de modo que a primeira chegada ao destino ocorre pelo menor caminho (CORMEN et al., 2012). O algoritmo foi adaptado para guardar todos os predecessores de cada vértice que estão à mesma distância, o que permite reconstruir os dois caminhos mínimos, e não apenas um.

Existem caminhos mais longos, como Ana → Bruno → Diego → Elisa → Felipe, com 4 arestas, mas nenhum com menos de 3. Assim, a distância entre Ana e Felipe é 3.

7 IMPLEMENTAÇÃO EM JAVA

O programa foi escrito em um único arquivo, `ConectaUDF.java`, compatível com Java 11 ou superior. Para executá-lo, basta compilar com `javac ConectaUDF.java` e rodar com `java ConectaUDF`. A saída reproduz todos os valores apresentados neste trabalho.

Os principais elementos do código são:

a) a classe `Grafo` guarda a lista de adjacência em um `LinkedHashMap<String, Set<String>>`, que mantém a ordem de inserção dos usuários;
b) o método `adicionarAmizade(a, b)` grava a aresta nos dois sentidos e rejeita laços, mantendo o grafo simples;
c) `grau(u)` retorna o tamanho do conjunto de amigos e `somaDosGraus()` soma todos eles;
d) `numArestas()` divide a soma dos graus por 2 e `densidade()` aplica a Equação 2;
e) `todosCaminhosMinimos()` executa a BFS e reconstrói os caminhos por retrocesso (backtracking), com complexidade O(V + E);
f) `recomendarAmizades()` percorre os pares que ainda não são amigos, calcula a interseção das listas de amigos e ordena as sugestões.

8 DESAFIO EM GRUPO: REDE AMPLIADA

Com a inclusão de 4 usuários e 7 novas amizades, a rede passou a ter 10 usuários e 15 amizades, e a densidade **caiu de 53,33% para 33,33%**.

8.1 Novos usuários e amizades

Foram adicionados os usuários Gabriel, Helena, Igor e Julia, com as seguintes amizades: Felipe e Gabriel, Elisa e Gabriel, Gabriel e Helena, Helena e Igor, Igor e Julia, Julia e Helena, e Julia e Gabriel. A lista de adjacência foi atualizada no programa por meio de uma cópia da rede original.

8.2 Graus na rede ampliada

A Tabela 2 apresenta os graus recalculados, ordenados do maior para o menor.

Tabela 2 – Graus dos usuários da rede ampliada

Usuário	Amigos diretos	Grau
Elisa	Carla, Diego, Felipe, Gabriel	4
Gabriel	Felipe, Elisa, Helena, Julia	4
Bruno	Ana, Carla, Diego	3
Carla	Ana, Bruno, Elisa	3
Diego	Bruno, Elisa, Felipe	3
Felipe	Diego, Elisa, Gabriel	3
Helena	Gabriel, Igor, Julia	3
Julia	Igor, Helena, Gabriel	3
Ana	Bruno, Carla	2
Igor	Helena, Julia	2

Fonte: elaborado pelo autor (2026).

8.3 Comparação da densidade

A Tabela 3 compara as métricas antes e depois da ampliação. Na rede ampliada, a densidade é D = (2 × 15) / (10 × 9) = 30/90 ≈ 0,3333.

Tabela 3 – Comparação entre a rede original e a ampliada

Medida	Rede original	Rede ampliada
Usuários (V)	6	10
Soma dos graus	16	30
Amizades (E)	8	15
Amizades possíveis	15	45
Densidade	53,33%	33,33%
Maior grau	3	4

Fonte: elaborado pelo autor (2026).

A densidade caiu porque as amizades quase dobraram (de 8 para 15),

enquanto as amizades possíveis triplicaram (de 15 para 45). O número de pares possíveis cresce com V^2, e cada pessoa só consegue manter um número limitado de amizades. Por isso, redes maiores tendem a ser mais esparsas, o que também ocorre em redes sociais reais.

8.4 Usuários com mais conexões

Elisa e Gabriel passaram a ter grau 4. Gabriel tem papel especial, pois funciona como ponte: todo caminho entre o grupo original e Helena, Igor e Julia passa por ele. Por exemplo, o menor caminho de Ana até Igor tem 5 arestas (Ana → Carla → Elisa → Gabriel → Helena → Igor). Se Gabriel saísse da rede, o grupo novo ficaria isolado.

8.5 Recomendação de novas amizades

A recomendação baseia-se no fechamento triádico: se A é amigo de B e B é amigo de C, é provável que A e C se conheçam. Para cada par que ainda não é amigo, o programa conta os amigos em comum e desempata pelo índice de Jaccard, que divide o número de amigos em comum pelo total de amigos distintos dos dois (LIBEN-NOWELL; KLEINBERG, 2007). A Tabela 4 mostra as cinco melhores sugestões.

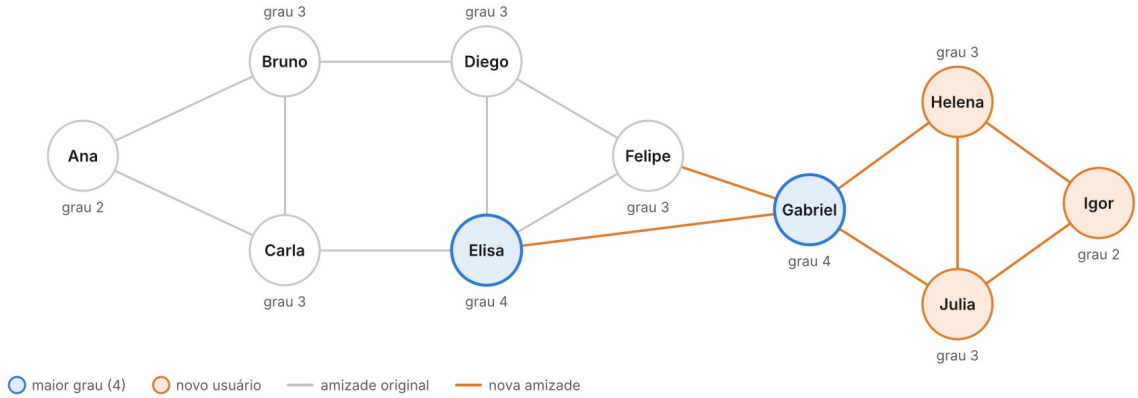
Tabela 4 – Sugestões de amizade na rede ampliada

Sugestão	Amigos em comum	Jaccard
Carla e Diego	Bruno, Elisa	0,50
Gabriel e Igor	Helena, Julia	0,50
Bruno e Elisa	Carla, Diego	0,40
Diego e Gabriel	Elisa, Felipe	0,40
Ana e Diego	Bruno	0,25

Fonte: elaborado pelo autor (2026).

Na prática, a rede exibiria primeiro uma mensagem como “Carla, você conhece Diego? Vocês têm 2 amigos em comum”.

Figura 2 – Grafo da rede ampliada



9 REFLEXÃO

Duas pessoas que ainda não são amigas, mas têm três amigos em comum, representam um sinal forte para sugerir a conexão. Quanto mais amigos compartilham, maior a chance de já se conhecerem ou de frequentarem os mesmos lugares. A rede pode transformar essa informação em uma pontuação, como o número de vizinhos em comum ou o índice de Jaccard, e exibir as sugestões de maior pontuação acompanhadas da justificativa, para que o usuário entenda por que as recebeu.

Ao analisar redes sociais, devem ser considerados os seguintes cuidados de privacidade:

a) a sugestão pode revelar quem são os amigos de uma pessoa, mesmo que ela tenha deixado sua lista de amigos privada;
b) o usuário deve poder ocultar sua lista de amigos e optar por não aparecer em sugestões;
c) sugestões podem expor vínculos sensíveis, como um ex-parceiro, um profissional de saúde ou alguém de quem a pessoa se afastou;
d) para calcular métricas como grau e densidade, não é preciso conhecer nomes, de modo que se podem usar identificadores e coletar apenas os dados necessários, conforme o princípio da necessidade da LGPD (BRASIL, 2018);
e) os dados do grafo não devem ser usados para finalidade diferente da

informada ao usuário, como a venda do grafo social a anunciantes.

APÊNDICE A – CÓDIGO-FONTE EM JAVA

```
import java.util.*;

/**
 * Projeto ConectaUDF - redes sociais com grafos
 * Disciplina: Algoritmos / Matematica Discreta / Ciencia de Dados
 *
 * Grafo NAO direcionado (amizade e reciproca), representado por
 * lista de adjacencia: Map<usuario, conjunto de amigos>.
 *
 * Compilar:  javac ConectaUDF.java
 * Executar:  java ConectaUDF
 */
public class ConectaUDF {

    // ========================================================================
    // Classe que representa o grafo
    // ========================================================================
    static class Grafo {
        // LinkedHashMap mantem a ordem de insercao (fica igual ao caderno na hora de
imprimir)
        // LinkedHashSet evita amizade duplicada (passo 4 do desenho: "nao desenhe duas
linhas")
        private final Map<String, Set<String>> adj = new LinkedHashMap<>();

        void adicionarUsuario(String nome) {
            adj.putIfAbsent(nome, new LinkedHashSet<>());
        }

        /** Adiciona a aresta nos DOIS sentidos, porque o grafo e nao direcionado. */
        void adicionarAmizade(String a, String b) {
            if (a.equals(b)) {
                throw new IllegalArgumentException("Grafo simples: sem laco (" + a + ")");
            }
            adicionarUsuario(a);
            adicionarUsuario(b);
            adj.get(a).add(b);
            adj.get(b).add(a);
        }

        Set<String> usuarios() {
            return adj.keySet();
        }

        Set<String> amigos(String u) {
            return adj.getOrDefault(u, Collections.emptySet());
        }

        boolean saoAmigos(String a, String b) {
            return amigos(a).contains(b);
        }

        int grau(String u) {
            return amigos(u).size();
        }

        int numVertices() {
            return adj.size();
        }

        int somaDosGraus() {
            int soma = 0;
            for (Set<String> amigos : adj.values()) {
                soma += amigos.size();
            }
            return soma;
        }

        /** Lema do aperto de maos: soma dos graus = 2E  =>  E = soma / 2 */
        int numArestas() {
            return somaDosGraus() / 2;
        }

        /** D = 2E / (V (V - 1)) para grafo simples nao direcionado */
```

```
        double densidade() {
            int v = numVertices();
            if (v < 2) return 0.0;
            return (2.0 * numArestas()) / (v * (v - 1.0));
        }

        int grauMaximo() {
            int max = 0;
            for (String u : adj.keySet()) max = Math.max(max, grau(u));
            return max;
        }

        List<String> usuariosComMaiorGrau() {
            int max = grauMaximo();
            List<String> lista = new ArrayList<>();
            for (String u : adj.keySet()) {
                if (grau(u) == max) lista.add(u);
            }
            return lista;
        }

        /**
         * BFS a partir da origem. Em grafo sem peso, a BFS encontra a menor
         * distancia (em numero de arestas). Guardo TODOS os pais de cada vertice
         * no menor caminho, assim da pra reconstruir todos os caminhos minimos.
         */
        List<List<String>> todosCaminhosMinimos(String origem, String destino) {
            Map<String, Integer> dist = new HashMap<>();
            Map<String, List<String>> pais = new HashMap<>();
            Deque<String> fila = new ArrayDeque<>();

            dist.put(origem, 0);
            pais.put(origem, new ArrayList<>());
            fila.add(origem);

            while (!fila.isEmpty()) {
                String atual = fila.poll();
                for (String viz : amigos(atual)) {
                    if (!dist.containsKey(viz)) {               // primeira vez que chego
                        dist.put(viz, dist.get(atual) + 1);
                        pais.put(viz, new ArrayList<>(List.of(atual)));
                        fila.add(viz);
                    } else if (dist.get(viz) == dist.get(atual) + 1) {
                        pais.get(viz).add(atual);               // outro caminho de mesmo
tamanho
                    }
                }
            }

            List<List<String>> caminhos = new ArrayList<>();
            if (!dist.containsKey(destino)) return caminhos;    // nao conectados
            montarCaminhos(destino, origem, pais, new LinkedList<>(), caminhos);
            return caminhos;
        }

        // Backtracking do destino ate a origem usando os pais da BFS
        private void montarCaminhos(String atual, String origem, Map<String, List<String>>
pais,
                                    LinkedList<String> parcial, List<List<String>> saida) {
            parcial.addFirst(atual);
            if (atual.equals(origem)) {
                saida.add(new ArrayList<>(parcial));
            } else {
                for (String p : pais.get(atual)) {
                    montarCaminhos(p, origem, pais, parcial, saida);
                }
            }
            parcial.removeFirst();
        }

        Set<String> amigosEmComum(String a, String b) {
            Set<String> comum = new LinkedHashSet<>(amigos(a));
            comum.retainAll(amigos(b));                         // intersecao
            return comum;
        }

        /**
```

```
     * Recomendacao por "amigos em comum" (common neighbors):
     * para cada par que AINDA NAO e amigo, conta quantos amigos os dois
     * compartilham. Quanto mais amigos em comum, mais forte a sugestao.
     * Tambem calculo o indice de Jaccard = |comum| / |uniao| para desempate.
     */
    List<Sugestao> recomendarAmizades() {
        List<String> lista = new ArrayList<>(adj.keySet());
        List<Sugestao> sugestoes = new ArrayList<>();

        for (int i = 0; i < lista.size(); i++) {
            for (int j = i + 1; j < lista.size(); j++) {
                String a = lista.get(i), b = lista.get(j);
                if (saoAmigos(a, b)) continue;

                Set<String> comum = amigosEmComum(a, b);
                if (comum.isEmpty()) continue;

                Set<String> uniao = new HashSet<>(amigos(a));
                uniao.addAll(amigos(b));
                double jaccard = (double) comum.size() / uniao.size();

                sugestoes.add(new Sugestao(a, b, comum, jaccard));
            }
        }

        sugestoes.sort((s1, s2) -> {
            int c = Integer.compare(s2.comum.size(), s1.comum.size());
            return c != 0 ? c : Double.compare(s2.jaccard, s1.jaccard);
        });
        return sugestoes;
    }

    Grafo copia() {
        Grafo g = new Grafo();
        for (String u : adj.keySet()) {
            g.adicionarUsuario(u);
            for (String v : adj.get(u)) g.adj.get(u).add(v);
        }
        return g;
    }
}

static class Sugestao {
    final String a, b;
    final Set<String> comum;
    final double jaccard;

    Sugestao(String a, String b, Set<String> comum, double jaccard) {
        this.a = a; this.b = b; this.comum = comum; this.jaccard = jaccard;
    }
}

// ========================================================================
// Funcoes de impressao (relatorio)
// ========================================================================
static void titulo(String t) {
    System.out.println();
    System.out.println("=".repeat(64));
    System.out.println(t);
    System.out.println("=".repeat(64));
}


static void imprimirListaAdjacencia(Grafo g) {
    for (String u : g.usuarios()) {
        System.out.printf("  %-8s -> %s%n", u, g.amigos(u));
    }
}

static void imprimirTabelaGraus(Grafo g) {
    System.out.printf("  %-8s | %-32s | %s%n", "Usuario", "Amigos diretos", "Grau");
    System.out.println("  " + "-".repeat(52));
    for (String u : g.usuarios()) {
        System.out.printf("  %-8s | %-32s | %4d%n",
                u, String.join(", ", g.amigos(u)), g.grau(u));
    }
}
```

```
    static void imprimirCalculos(Grafo g) {
        int v = g.numVertices();
        int soma = g.somaDosGraus();
        int e = g.numArestas();
        double d = g.densidade();

        System.out.println("  Soma dos graus ........ " + soma);
        System.out.println("  Arestas = " + soma + " / 2 ... " + e);
        System.out.println("  Vertices (V) .......... " + v);
        System.out.printf("  D = 2 x %d / (%d x (%d - 1)) = %d / %d%n",
                e, v, v, 2 * e, v * (v - 1));
        System.out.printf("  Densidade decimal ..... %.4f%n", d);
        System.out.printf("  Densidade percentual .. %.2f%%%n", d * 100);
        System.out.println("  Maior grau ............ " + g.grauMaximo()
                + " -> " + g.usuariosComMaiorGrau());
    }

    static void imprimirCaminhos(Grafo g, String origem, String destino) {
        List<List<String>> caminhos = g.todosCaminhosMinimos(origem, destino);
        if (caminhos.isEmpty()) {
            System.out.println("  Nao existe caminho entre " + origem + " e " + destino);
            return;
        }
        int arestas = caminhos.get(0).size() - 1;
        System.out.println("  Menor caminho tem " + arestas + " arestas. Caminhos minimos
encontrados: "
                + caminhos.size());
        for (List<String> c : caminhos) {
            System.out.println("    " + String.join(" -> ", c));
        }
    }

    static void imprimirRecomendacoes(Grafo g, int limite) {
        List<Sugestao> sugestoes = g.recomendarAmizades();
        if (sugestoes.isEmpty()) {
            System.out.println("  Nenhuma sugestao (ninguem tem amigos em comum).");
            return;
        }
        int n = 0;
        for (Sugestao s : sugestoes) {
            if (n++ == limite) break;
            System.out.printf("  %-7s <-> %-7s | %d em comum %s | Jaccard = %.2f%n",
                    s.a, s.b, s.comum.size(), s.comum, s.jaccard);
        }
    }

    // ========================================================================
    // Programa principal
    // ========================================================================
    public static void main(String[] args) {

        // ---------------- Rede original (secao 3 do caderno) ----------------
        Grafo rede = new Grafo();
        String[][] amizades = {
                {"Ana", "Bruno"},
                {"Ana", "Carla"},
                {"Bruno", "Carla"},
                {"Bruno", "Diego"},
                {"Carla", "Elisa"},
                {"Diego", "Elisa"},
                {"Diego", "Felipe"},
                {"Elisa", "Felipe"}
        };
        for (String[] par : amizades) rede.adicionarAmizade(par[0], par[1]);

        titulo("PARTE 1 - REDE ORIGINAL CONECTAUDF");

        System.out.println("\n[Secao 8] Lista de adjacencia:");
        imprimirListaAdjacencia(rede);

        System.out.println("\n[Secao 5] Graus:");
        imprimirTabelaGraus(rede);

        System.out.println("\n[Secoes 6 e 7] Arestas e densidade:");
        imprimirCalculos(rede);
```

```
        System.out.println("\n[Secao 9] Caminhos de Ana ate Felipe (BFS):");
        imprimirCaminhos(rede, "Ana", "Felipe");

        System.out.println("\n[Extra] Sugestoes de amizade na rede original:");
        imprimirRecomendacoes(rede, 5);

        // ---------------- Rede ampliada (secao 10 - desafio em grupo) -------
        Grafo ampliada = rede.copia();
        String[][] novasAmizades = {
                {"Felipe", "Gabriel"},
                {"Elisa", "Gabriel"},
                {"Gabriel", "Helena"},
                {"Helena", "Igor"},
                {"Igor", "Julia"},
                {"Julia", "Helena"},
                {"Julia", "Gabriel"}
        };
        for (String[] par : novasAmizades) ampliada.adicionarAmizade(par[0], par[1]);

        titulo("PARTE 2 - REDE AMPLIADA (DESAFIO EM GRUPO)");
        System.out.println("  Novos usuarios: Gabriel, Helena, Igor, Julia");
        System.out.println("  Novas amizades: " + novasAmizades.length);

        System.out.println("\nLista de adjacencia atualizada:");
        imprimirListaAdjacencia(ampliada);

        System.out.println("\nGraus:");
        imprimirTabelaGraus(ampliada);

        System.out.println("\nArestas e densidade:");
        imprimirCalculos(ampliada);

        System.out.println("\nComparacao de densidade:");
        System.out.printf("  Antes:  %.2f%%  (V=%d, E=%d)%n",
                rede.densidade() * 100, rede.numVertices(), rede.numArestas());
        System.out.printf("  Depois: %.2f%%  (V=%d, E=%d)%n",
                ampliada.densidade() * 100, ampliada.numVertices(), ampliada.numArestas());

        System.out.println("\nCaminho de Ana ate Igor (atravessando a rede nova):");
        imprimirCaminhos(ampliada, "Ana", "Igor");

        System.out.println("\nTop 5 sugestoes de amizade (amigos em comum):");
        imprimirRecomendacoes(ampliada, 5);
    }
}
```